Reproducing Results on the Andrews–Curtis Conjecture: Classical Search and PPO

AC-Solver Project

April 2026

Abstract

This document provides a self-contained account of our reproduction experiments on the Andrews–Curtis conjecture using the Miller–Schupp dataset (1190 balanced presentations). We describe (1) the classical search algorithms (greedy and breadth-first) including pseudocode, exact reproduction commands, and results; (2) a Proximal Policy Optimisation (PPO) agent trained for 10^8 environment steps, its architecture, hyperparameters, training dynamics, and evaluation; and (3) a comparative analysis showing that the PPO agent learns genuine but shallow problem structure, solving 2.7% of instances at best versus 46.6% for greedy search. All commands needed to reproduce every result are collected in Appendix A.

Contents

1 Introduction

1.1 The Andrews–Curtis Conjecture

The *Andrews–Curtis conjecture* (1965) asserts that every balanced presentation of the trivial group can be reduced to the standard trivial presentation $\langle x, y \mid x, y\rangle$ using a finite sequence of *AC moves*: Nielsen transformations on relators and conjugation by generators. Despite extensive computational and theoretical effort, the conjecture remains open.

Computationally, the problem reduces to a *graph search*: each node is a balanced presentation $\langle x, y \mid r_0, r_1\rangle$, each edge is one of 12 AC moves, and the goal is to reach a trivial node (total relator length = 2).

1.2 The Miller–Schupp Dataset

We use the Miller–Schupp family of presentations, parameterised by an integer $n \geq 1$ and a balanced word w over $\{x, x^{-1}, y, y^{-1}\}$:

$$\mathrm{MS}(n, w) \;=\; \langle x, y \mid x^{-1}\, y^{n}\, x\, y^{-(n+1)},\;\; x^{-1}\, w\rangle.$$

With $n \in \{1, \ldots, 7\}$ and $|w| \in \{1, \ldots, 7\}$, after removing cyclic permutations and applying free/-cyclic reduction, the dataset contains **1190 presentations**. These are stored in `ac_solver/search/miller_sch` one per line.

1.3 Presentation Encoding

Each presentation is encoded as a NumPy `int8` array of length $2 \times$ `max_relator_length`. The first half stores r_0 and the second half stores r_1, both right-padded with zeros:

Value	Meaning
$+1$	generator x
-1	generator x^{-1}
$+2$	generator y
-2	generator y^{-1}
0	padding (unused position)

For the Miller–Schupp dataset, `max_relator_length` $= 36$ (derived from $\max_n(4n{+}2)$ for $n \leq 7$), giving observation vectors of dimension 72.

1.4 Success Criterion

A presentation is *solved* (trivialised) when the sum of both relators' non-zero lengths equals 2, i.e. each relator is a single generator or its inverse. For example, $\langle x, y \mid x, y^{-1}\rangle$ is trivial.

2 The 12 AC Moves

At each state, there are 12 possible actions: 4 concatenations and 8 conjugations. After every move, *free reduction* is applied automatically (cancelling adjacent inverse pairs such as $x \cdot x^{-1}$). If the resulting relator would exceed `max_relator_length`, the move is rejected and the state is returned unchanged (a "no-op").

Free Reduction. After every move, adjacent inverse pairs are cancelled iteratively. For example, if a concatenation produces $\ldots x \cdot x^{-1} \ldots$, the pair is removed. This is applied until no more cancellations are possible.

Table 1: The 12 AC moves. Here $x_1 = x$, $x_2 = y$.

ID	Type	Operation	Description
0	Concat	$r_1 \leftarrow r_1 \cdot r_0$	Append r_0 to r_1
1	Concat	$r_0 \leftarrow r_0 \cdot r_1^{-1}$	Append r_1^{-1} to r_0
2	Concat	$r_1 \leftarrow r_1 \cdot r_0^{-1}$	Append r_0^{-1} to r_1
3	Concat	$r_0 \leftarrow r_0 \cdot r_1$	Append r_1 to r_0
4	Conj	$r_1 \leftarrow x^{-1} \cdot r_1 \cdot x$	Conjugate r_1 by x^{-1}
5	Conj	$r_0 \leftarrow y^{-1} \cdot r_0 \cdot y$	Conjugate r_0 by y^{-1}
6	Conj	$r_1 \leftarrow y^{-1} \cdot r_1 \cdot y$	Conjugate r_1 by y^{-1}
7	Conj	$r_0 \leftarrow x \cdot r_0 \cdot x^{-1}$	Conjugate r_0 by x
8	Conj	$r_1 \leftarrow x \cdot r_1 \cdot x^{-1}$	Conjugate r_1 by x
9	Conj	$r_0 \leftarrow y \cdot r_0 \cdot y^{-1}$	Conjugate r_0 by y
10	Conj	$r_1 \leftarrow y \cdot r_1 \cdot y^{-1}$	Conjugate r_1 by y
11	Conj	$r_0 \leftarrow x^{-1} \cdot r_0 \cdot x$	Conjugate r_0 by x^{-1}

Move Rejection. If the resulting relator would have length > 36 (after free reduction), the move returns the original state unchanged. This is a significant source of "no-op" steps: approximately 69% of actions during PPO training produce no state change.

3 Classical Search Algorithms

Both algorithms explore the same graph of balanced presentations. They differ only in the order states are expanded: greedy uses a priority queue ordered by total relator length; BFS uses a FIFO queue ordered by insertion time (depth).

3.1 Greedy Search

Core idea. Always expand the state with the *shortest total presentation length* first, using a min-heap. This is a heuristic: it assumes shorter presentations are closer to the trivial state.

Algorithm 1 Greedy Search for AC Trivialisation

Require: Presentation P, max nodes N
Ensure: True/False, path, visited count

$L \leftarrow \text{MaxRelatorLength}(P)$
$\ell_0, \ell_1 \leftarrow \text{WordLengths}(P)$
$visited \leftarrow \{P\}$
$heap \leftarrow [(\ell_0 + \ell_1,\ 0,\ P,\ (\ell_0, \ell_1),\ [(-1, \ell_0 + \ell_1)])]$ ▷ priority, depth, state, lengths, path
while $heap \neq \emptyset$ **and** $|visited| < N$ **do**
    $(\_, d, S, w, path) \leftarrow \text{HeapPop}(heap)$
    for $a \in \{0, 1, \ldots, 11\}$ **do**
        $S', w' \leftarrow \text{ACMove}(a, S, L, w)$
        $\ell' \leftarrow w'_0 + w'_1$
        if $\ell' = 2$ **then**
            return True, $path \parallel [(a, \ell')]$, $|visited|$
        end if
        if $S' \notin visited$ **then**
            $visited \leftarrow visited \cup \{S'\}$
            $\text{HeapPush}(heap,\ (\ell',\ d+1,\ S',\ w',\ path \parallel [(a, \ell')]))$
        end if
    end for
end while
return False, $path$, $|visited|$

Tie-breaking. When two states share the same total length, the heap breaks ties by (i) path length (prefer fewer moves from start), then (ii) lexicographic comparison of the state tuple.

Properties.

- **No shortest-path guarantee.** Greedy may find a solution via a longer path because it chases locally short states.
- **Typically faster.** By focusing on short states, it often explores fewer nodes than BFS.
- **Per-step cost:** $O(\log n)$ for heap operations.

Source code. `ac_solver/search/greedy.py`, function `greedy_search()`.

3.2 Breadth-First Search (BFS)

Core idea. Expand states strictly level by level—all states reachable in d moves before any state at depth $d+1$—using a FIFO queue. Because every AC move has unit cost, BFS *guarantees finding the shortest path* (fewest moves).

Algorithm 2 Breadth-First Search for AC Trivialisation

Require: Presentation P, max nodes N
Ensure: True/False, path, visited count

$L \leftarrow \text{MaxRelatorLength}(P)$
$visited \leftarrow \{P\}$
$queue \leftarrow \text{Deque}([(P,\ [(-1, \text{TotalLength}(P))])])$
while $queue \neq \emptyset$ **and** $|visited| < N$ **do**
  $(S, path) \leftarrow \text{PopLeft}(queue)$
  $w \leftarrow \text{WordLengths}(S)$ ▷ Recomputed from current state
  for $a \in \{0, 1, \ldots, 11\}$ **do**
    $S', w' \leftarrow \text{ACMove}(a, S, L, w)$
    $\ell' \leftarrow w'_0 + w'_1$
    if $\ell' = 2$ **then**
      return True, $path \parallel [(a, \ell')]$, $|visited|$
    end if
    if $S' \notin visited$ **then**
      $visited \leftarrow visited \cup \{S'\}$
      $\text{Append}(queue,\ (S',\ path \parallel [(a, \ell')]))$
    end if
  end for
end while
return False, None, $|visited|$

Why BFS guarantees the shortest path. All states at depth d are expanded before any at depth $d + 1$. The first time BFS reaches a trivial state, it must be via a minimum-length path—all shorter paths have already been fully explored.

Properties.

- **Shortest-path guarantee.** The first solution found has the fewest AC moves of any solution within the node budget.
- **Typically slower.** Exhaustive level-by-level expansion explores many more nodes than greedy.
- **Per-step cost:** $O(1)$ for deque append/popleft.

Source code. `ac_solver/search/breadth_first.py`, function `bfs()`.

3.3 Side-by-Side Comparison

Table 2: Greedy vs. BFS comparison.

Aspect	**Greedy**	**BFS**
Data structure	Min-heap (priority queue)	FIFO queue (deque)
Expansion order	Shortest total length first	By depth (oldest first)
Shortest path?	No	Yes (guaranteed)
Typical speed	Faster (fewer nodes explored)	Slower (exhaustive per level)
Per-step cost	$O(\log n)$	$O(1)$

3.4 Classical Search Results

Both algorithms were run on all 1190 Miller–Schupp presentations with a node budget of 10^6.

Table 3: Classical search results (node budget = 10^6).

Algorithm	Solved	Solve Rate	Paper Reference
Greedy	554 / 1190	46.6%	533 / 1190 (44.8%)
BFS	278 / 1190	23.4%	278 / 1190 (23.4%)

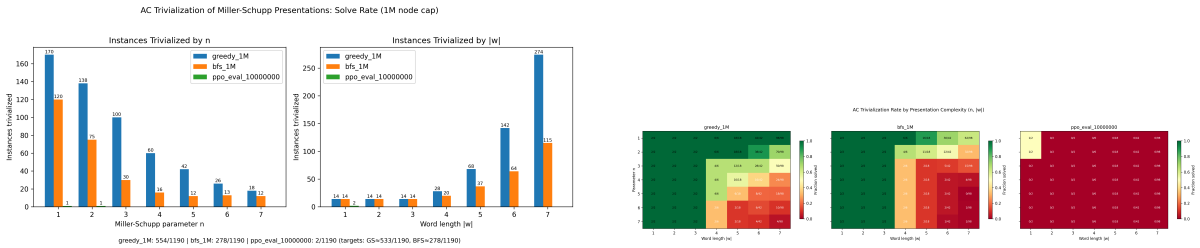
Our BFS result matches the paper exactly. Greedy search solves 21 more instances than reported in the paper (554 vs. 533); this small discrepancy likely reflects minor implementation differences in tie-breaking or free reduction.

How to reproduce.

```
# Greedy search on full dataset (1M node budget, 4 workers)
python -m ac_solver.search.miller_schupp.evaluate \
    --search-fn greedy --max-nodes 1000000 \
    --workers 4 --full --output results/greedy_1M.csv

# BFS on full dataset
python -m ac_solver.search.miller_schupp.evaluate \
    --search-fn bfs --max-nodes 1000000 \
    --workers 4 --full --output results/bfs_1M.csv
```

Results by difficulty. Figure 1 shows the solve rate broken down by the Miller–Schupp parameters n (relator complexity) and $|w|$ (word length), and a heatmap of solve rate by $(n, |w|)$.



(a) Solve rate by n and $|w|$.

(b) Heatmap: solve rate by $(n, |w|)$.

Figure 1: Classical search results on the Miller–Schupp dataset.

4 PPO Reproduction

4.1 Architecture

The agent uses separate actor and critic networks, each a 2-layer feed-forward MLP:

Table 4: Actor–critic network architecture.

Component	Architecture	Init
Actor	$72 \to 512$ (Tanh) $\to 512$ (Tanh) $\to 12$	Orthogonal ($\sqrt{2}$ hidden, 0.01 output)
Critic	$72 \to 512$ (Tanh) $\to 512$ (Tanh) $\to 1$	Orthogonal ($\sqrt{2}$ hidden, 1.0 output)

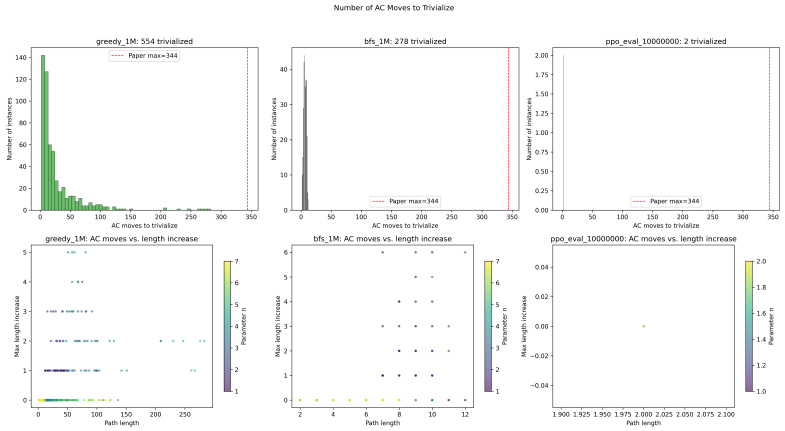


Figure 2: Distribution of path lengths (number of AC moves) for solved instances.

- **Input dimension:** $72 = 2 \times 36$ (two relators, each padded to `max_relator_length`).
- **Output:** 12 logits (one per AC move) for the actor; 1 scalar value for the critic.
- **Action selection:** Categorical(logits) during training (stochastic); $\arg\max$(logits) during greedy evaluation (deterministic).
- **Weight initialisation:** Orthogonal with gain $\sqrt{2}$ for hidden layers, 0.01 for the actor output layer (near-uniform initial policy), and 1.0 for the critic output layer.

Source code. `ac_solver/agents/ppo_agent.py`.

4.2 Hyperparameters

Table 5: PPO hyperparameters: paper vs. our Mac reproduction.

Parameter	Paper	Mac Repro	Notes
`total_timesteps`	10^8	10^8	Same
`num_envs`	28	8	Hardware constraint
`num_steps`	200	200	Same (episode horizon)
`batch_size`	5,600	1,600	= `num_envs` × `num_steps`
`minibatch_size`	1,400	400	= `batch_size`/4
`num_minibatches`	4	4	Same
`update_epochs`	1	1	Single pass per batch
`learning_rate`	10^{-4}	$\to 0$ (linear)	Same schedule
γ	0.999	0.999	Discount factor
λ_{GAE}	0.95	0.95	GAE parameter
ϵ_{clip}	0.2	0.2	PPO clip coefficient
c_{ent}	0.01	0.01	Entropy bonus
c_{vf}	0.5	0.5	Value loss coefficient
`max_grad_norm`	0.5	0.5	Gradient clipping
$\text{KL}_{\text{target}}$	0.01	0.01	Early stop threshold
`repeat_solved_prob`	0.25	0.25	Curriculum revisit prob
PPO updates	17,857	62,500	= `total_timesteps`/`batch_size`

The Mac run gets 3.5× more PPO updates, but each with a 3.5× smaller (and noisier) batch.

4.3 Reward Pipeline

The reward undergoes three transformations before reaching the PPO update:

1. **Raw reward** (from `ACEnv`):
   - Success: +14,400 (= horizon × `max_relator_length` × 2 = 200 × 36 × 2)
   - Per-step: $-\sum$(lengths) (typically −5 to −70)
2. **Reward clipping** to $[-10, 1000]$:
   - Success: +14,400 → +1,000 (**93% of signal discarded**)
   - Per-step: $\max(-10, -\sum(\text{lengths}))$
3. **Division by** `max_reward` (= 14,400):
   - Success: +1,000/14,400 = +0.069
   - Per-step: −10/14,400 = −0.00069

The net effect is a 200× **attenuation** of the success signal. This is the most significant concern with the current setup (see Section 5).

4.4 Curriculum

- **Round 1:** Cycle through all 1190 presentations in order, one per episode reset.
- **Round 2+:** With probability 0.25, revisit a solved presentation; otherwise pick an unsolved one.
- All 8 parallel environments independently track their position.

4.5 Training Execution

Training ran in two phases due to a session interruption:

Table 6: Training execution phases.

Phase	Steps	Updates	Duration
Initial	0 → 38.8M	1 → 24,285	∼24 hours
Resumed	38.7M → 100M	24,201 → 62,500	∼36 hours
Total	10^8	62,500	∼60 hours

Checkpoints were saved at **1M, 5M, 10M, 20M** (initial run) and **50M, 100M** (resumed run).

4.6 Training Metrics

4.6.1 Metric Trajectory

Table 7: Key metrics at checkpoint steps. "Solved" = cumulative presentations marked solved during training (stochastic).

Step	Entropy	Logit Gap	Top-1 Prob	Expl. Var	Clip Frac	Solved
1,600 (start)	2.485	0.003	0.084	−0.920	0.0	0/1190
1,000,000	2.483	0.035	0.093	−2.062	0.0	116/1190
5,000,000	2.454	0.172	0.130	−1.332	0.0	843/1190
10,000,000	1.919	1.162	0.397	+0.397	0.0	1190/1190
20,000,000	1.363	1.667	0.591	+0.510	0.0	1190/1190
50,000,000	∼1.2	∼1.4	∼0.58	∼0.6	0.0	1190/1190
100,000,000	1.253	1.415	0.601	+0.656	0.0	1190/1190

Table 8: Phase-averaged training metrics.

Phase	Avg Entropy	Avg Top-1	Avg Expl. Var	Avg Noop	Train Solve Rate
0–1M	2.484	0.089	−1.832	0.604	2.5%
1–5M	2.470	0.109	−1.834	0.607	4.7%
5–10M	2.283	0.229	−0.559	0.632	49.3%
10–20M	1.664	0.482	+0.371	0.674	88.0%
20–50M	1.254	0.592	+0.690	0.666	94.1%
50–100M	1.235	0.594	+0.734	0.656	94.5%

4.6.2 Phase-Averaged Metrics

4.6.3 Evidence of Network Change

Four independent metrics confirm the policy parameters changed significantly:

1. **Entropy:** 2.485 → 1.253. A uniform policy over 12 actions has entropy $\ln(12) \approx 2.485$. The final value indicates the policy concentrates probability mass on a subset of actions.
2. **Top-1 probability:** 8.4% → 60.1%. A uniform policy assigns $1/12 \approx 8.3\%$ to each action.
3. **Logit gap:** 0.003 → 1.415. The actor developed distinct preferences between actions.
4. **Explained variance:** −0.92 → +0.66. The critic learned to predict which states lead to success.

4.6.4 Why `clipfrac` = 0 Throughout

With `update_epochs` = 1, each batch is processed in a single pass through 4 minibatches. The first minibatch sees an *unchanged* network (ratio = 1.0 exactly). Subsequent minibatches see a network shifted by 1–3 gradient steps, but with:

- normalised advantages of std ≈ 0.03,
- gradient norm clipped to 0.5,
- learning rate $\leq 10^{-4}$,

the policy ratio stays well within $[0.8, 1.2]$. This is *expected behaviour*, not a bug. Over 62,500 updates, these small changes accumulate into the large entropy reduction observed.

4.6.5 Action Distribution Shift

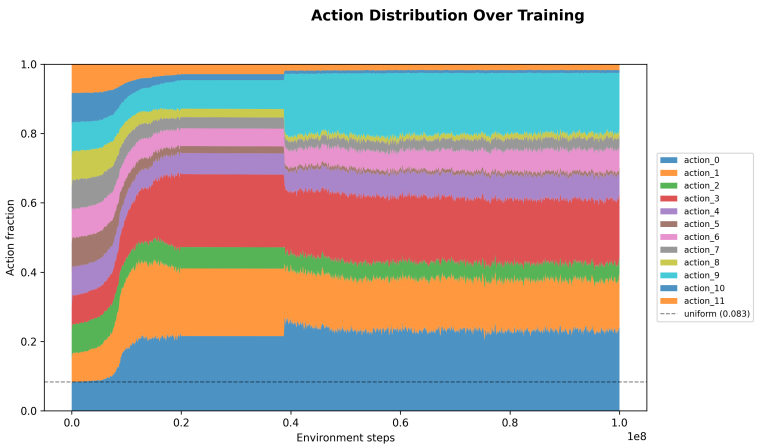


Figure 3: Action frequency distribution over training. The policy shifts from uniform (∼8.3% each) to concentrated, favouring concatenation moves (actions 0, 1, 3) over conjugations.

The policy learned to favour concatenation moves (actions 0–3), which are the primary mechanism for reducing relator lengths. Certain conjugations (actions 5, 8, 10, 11) are used very rarely ($< 3\%$ each).

4.6.6 Training Curves

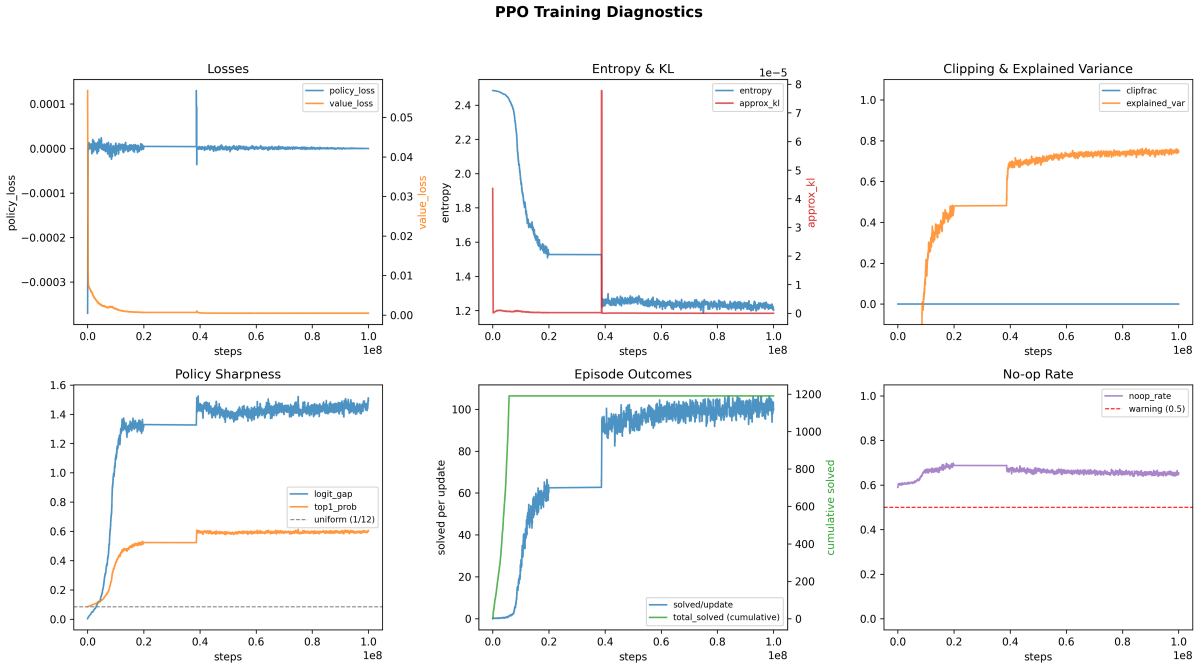


Figure 4: Training diagnostics (2×3 grid). **Top:** losses, entropy/KL, clipping/explained variance. **Bottom:** policy sharpness (logit gap, top-1 prob), episode outcomes, noop rate.

4.7 Evaluation Results

We evaluated 6 checkpoints across 3 modes on the full 1190-instance dataset with horizon = 200:

- **Greedy (deterministic):** action = $\arg\max$(logits)
- **Stochastic (×1):** action $\sim$ Categorical(logits)
- **Stochastic (×5, best-of-5):** 5 independent rollouts, report best

Table 9: PPO evaluation results (solved out of 1190 instances).

Checkpoint	Greedy (det)	Stoch ×1	Stoch ×5
1M	0 (0.0%)	2 (0.2%)	7 (0.6%)
5M	5 (0.4%)	6 (0.5%)	15 (1.3%)
10M	7 (0.6%)	12 (1.0%)	30 (2.5%)
20M	7 (0.6%)	10 (0.8%)	30 (2.5%)
50M	**6 (0.5%)**	**16 (1.3%)**	**32 (2.7%)**
100M	6 (0.5%)	13 (1.1%)	29 (2.4%)

Peak performance is at 50M steps, not 100M—the linear LR→0 schedule degraded the policy in the second half of training.

Table 10: PPO vs. classical baselines.

Algorithm	Solved	Solve Rate
Greedy search (10^6 nodes)	554 / 1190	46.6%
BFS (10^6 nodes)	278 / 1190	23.4%
PPO 50M greedy (best det)	6 / 1190	0.5%
PPO 50M best-of-5 stoch (best overall)	32 / 1190	2.7%

4.7.1 Comparison with Classical Baselines

Classical greedy search (a simple heuristic with no learning) solves 17× **more** presentations than the best PPO configuration.

4.7.2 PPO vs. Random Baseline

To determine whether PPO learned anything useful, we compare the best PPO checkpoint (50M best-of-5) against a near-random policy (1M best-of-5):

Table 11: PPO vs. near-random policy.

Policy	Best-of-5 Solved	Interpretation
Near-random (1M)	7 / 1190 (0.6%)	Random walk baseline
PPO 50M	32 / 1190 (2.7%)	4.6× improvement
Overlap	4	Solved by both
PPO-only	28	Genuine learning
Random-only	3	Noise

The PPO policy is **significantly better than random** (4.6×), confirming genuine learning. However, it solves only the easiest instances (short initial lengths, low n).

How to reproduce PPO evaluation.

```
# Deterministic evaluation on full dataset
python -m ac_solver.agents.evaluate \
    --checkpoint out/<run_dir>/ckpt_50000000.pt \
    --horizon 200 --full \
    --output results/ppo_50M_det.csv

# Stochastic best-of-5
python -m ac_solver.agents.evaluate \
    --checkpoint out/<run_dir>/ckpt_50000000.pt \
    --horizon 200 --full --stochastic --num-rollouts 5 \
    --eval-seed 42 --output results/ppo_50M_sto5.csv
```

5 Diagnosis and Recommendations

5.1 Why PPO Underperformed

1. **Catastrophic reward attenuation.** The success reward (+14,400) is clipped to +1,000 then divided by 14,400, yielding a final signal of +0.069—a 200× reduction. The policy gradient is dominated by noise for most updates.

2. **Insufficient data reuse.** With `update_epochs` = 1, each batch of experience is used exactly once. Standard PPO uses 3–10 epochs.

3. **Batch size too small.** The 1,600-sample batch (vs. paper's 5,600) yields 3.5× noisier gradient estimates per update.

4. **LR→0 degradation.** Linear annealing to zero means the last ∼10% of training has negligible learning. The 50M checkpoint outperforms the 100M checkpoint.

5. **Misleading training metrics.** The 100% training solve rate (by 10M steps) reflects stochastic exploration during 200-step rollouts, not learned policy quality. At 2.5% solve rate, even a near-random policy "solves" many presentations via random walks.

6. **High noop rate (∼69%).** Two-thirds of actions produce no state change, wasting the action budget.

5.2 Recommended Next Steps (Ranked by Expected Impact)

1. **Fix reward scaling.** Either remove the `clip_rewards` step (letting the full +14,400 signal through) or use running normalisation via `gym.wrappers.NormalizeReward` instead of the fixed division by `max_reward`.

2. **Increase `update_epochs` to 4.** Extract 4× more learning per batch. The `target_kl` = 0.01 early stop prevents overfitting.

3. **Increase `num_envs`** to 16 or 28 if hardware permits, improving gradient quality and presentation diversity per batch.

4. **Use cosine annealing with `min_lr_frac` = 0.1.** Maintain a learning rate floor of 10^{-5} instead of decaying to zero.

5. **Add action masking.** Prevent the agent from selecting moves that would exceed `max_relator_length`, reducing the effective action space and noop rate.

6. **Monitor greedy eval during training.** Add periodic greedy evaluation (e.g. every 10^6 steps on a small subset) as the ground-truth metric, since training solve rate is misleading.

6 Conclusions

Was the training code correct? Yes, mechanically. The PPO implementation is standard and bug-free. All hyperparameters match or closely approximate the paper's configuration (adjusted for hardware). The persistent `clipfrac` = 0 is a consequence of conservative hyperparameters (`update_epochs` = 1, small reward signal), not a bug.

Did the policy learn? Weakly. The best configuration (50M, best-of-5 stochastic) solves 32/1190 instances (2.7%)—a 4.6× improvement over a near-random baseline (7/1190), with 28 instances solved exclusively by the trained policy. However, this is 17× worse than classical greedy search (554/1190).

Classical search remains far superior. Greedy search, a simple heuristic requiring no training, solves 46.6% of the dataset. BFS, which guarantees optimal path length, solves 23.4%. Both vastly outperform the PPO agent, suggesting that the current RL formulation does not effectively capture the combinatorial structure of the Andrews–Curtis problem.

A Reproduction Commands

A.1 Classical Search

```
# Greedy search: full dataset, 1M node budget
python -m ac_solver.search.miller_schupp.evaluate \
    --search-fn greedy --max-nodes 1000000 \
    --workers 4 --full --output results/greedy_1M.csv

# BFS: full dataset, 1M node budget
python -m ac_solver.search.miller_schupp.evaluate \
    --search-fn bfs --max-nodes 1000000 \
    --workers 4 --full --output results/bfs_1M.csv

# Resume an interrupted run
python -m ac_solver.search.miller_schupp.evaluate \
    --search-fn greedy --max-nodes 1000000 \
    --workers 8 --full --output results/greedy_1M.csv --resume

# Run on a single presentation
python ac_solver/search/greedy.py --preset AK2 --max-nodes 100000

# Run BFS on a specific presentation string
python ac_solver/search/breadth_first.py \
    --presentation "1,1,-2,-2,-2,0,0,1,2,1,-2,-1,-2,0" \
    --max-nodes 500000
```

A.2 PPO Training

```
# Train with paper-faithful Mac preset (100M steps, 8 envs)
python -m ac_solver.agents.ppo \
    --preset paper_faithful_mac \
    --seed 1 \
    --exp-name paper_faithful_mac_s1

# Resume from checkpoint
python -m ac_solver.agents.ppo \
    --preset paper_faithful_mac \
    --seed 1 \
    --exp-name paper_faithful_mac_s1_resumed \
    --resume --checkpoint-dir out/<previous_run_dir>
```

A.3 PPO Evaluation

```
# Deterministic (greedy) evaluation
python -m ac_solver.agents.evaluate \
    --checkpoint out/<run_dir>/ckpt_50000000.pt \
    --horizon 200 --full \
    --output results/ppo_50M_det.csv

# Stochastic single rollout
python -m ac_solver.agents.evaluate \
    --checkpoint out/<run_dir>/ckpt_50000000.pt \
    --horizon 200 --full --stochastic \
    --output results/ppo_50M_sto.csv
```

```
# Best-of-5 stochastic
python -m ac_solver.agents.evaluate \
    --checkpoint out/<run_dir>/ckpt_50000000.pt \
    --horizon 200 --full --stochastic --num-rollouts 5 \
    --eval-seed 42 --output results/ppo_50M_sto5.csv

# Best-of-32 (high rollout count)
python -m ac_solver.agents.evaluate \
    --checkpoint out/<run_dir>/ckpt_100000000.pt \
    --horizon 200 --full --stochastic --num-rollouts 32 \
    --eval-seed 42 --output results/ppo_100M_sto32.csv
```

A.4 Plotting

```
# Training curves from metrics CSV
python -m ac_solver.agents.plot_training_curves \
    --csv report/metrics_combined.csv --output-dir report/

# Classical search result figures
python -m ac_solver.search.miller_schupp.plot_results \
    --csvs results/greedy_1M.csv results/bfs_1M.csv \
    --output-dir results/figures/
```

B CSV Output Schema

B.1 Classical Search CSV

Each row corresponds to one presentation instance:

Field	Description
`instance_id`	Integer index (0–1189)
`n`	Miller–Schupp parameter n
`lenw`	Word length $\|w\|$
`algorithm`	`greedy` or `bfs`
`max_nodes`	Node budget
`solved`	`True`/`False`
`visited_nodes`	Unique states explored
`path_length`	Number of AC moves in solution
`initial_total_length`	Starting $\|r_0\| + \|r_1\|$
`max_intermediate_length`	Peak length during search
`length_increase`	max_intermediate − initial
`wallclock_seconds`	Elapsed time

B.2 PPO Evaluation CSV

Same schema as classical search, with `algorithm` set to `ppo` (deterministic) or `ppo_stochastic` (sampling).

C Checkpoint Inventory

File	Steps	Directory
ckpt_1000000.pt	1M	out/paper_faithful_mac_s1_ppo-ffn-nodes...f6f8985b...
ckpt_5000000.pt	5M	(same)
ckpt_10000000.pt	10M	(same)
ckpt_20000000.pt	20M	(same)
ckpt_50000000.pt	50M	out/paper_faithful_mac_s1_resumed...c3f0ba19...
ckpt_100000000.pt	100M	(same)